

树上问题

最近几年，noip比赛的难题难度在逐年提升，题目类型也有以前的偏知识变为了偏思维，查考范围也有主要考察序列问题变为考察树上问题。而以前的很多算法，特别是很多暴力算法都是针对序列的，在考场上难以发挥它的作用。所以我们今天研究下树上的带技巧性的暴力做法。

我们前面学过的暴力主要就是分块为主，比如分块，莫队，块状链表等等。那么如果涉及的是树上问题，那么我们可以对树也进行分块吗？

H D U - 4 3 6 6

题目

肖恩拥有一家公司，他是老板。每一个员工有一个上司。每个员工都有忠诚和能力。有时肖恩会解雇一名员工。然后，一个被解雇的人的下属将取代他的能力比他高，并有最高的忠诚度的公司。西恩想知道谁来接替那个被解雇的人。

输入

在第一行中，数字T表示测试用例的数量。那么对于每一种情况，第一行包含2个数字n,m ($2 \leq n, m \leq 50000$)，表示公司有n个人包括Sean,m是Sean的查询次数。员工编号从1到n-1,Sean的编号是0。按照n-1行，第i行($1 \leq i \leq n-1$)包含3个整数a,b,c($0 \leq a \leq n-1, 0 \leq b, c \leq 1000000$)，表示第i个员工的上级序列号，第i个员工的忠诚度和能力。每个员工的序列号都大于他的上级，每个员工都有不同的忠诚度。然后是m行查询。每行只有一个数字表明应该解雇谁的序列号。

输出

每个查询打印一个数字:谁将代替失去工作的人的序列号，如果没有人可以代替他，打印-1。

树上问题

理解题意之后，该题本质上就是求每棵子树中比子树的根节点能力值大并且最忠诚的那个点的编号。

对于树上子树统计问题，我们要条件性的想到几种常见的解决办法。

第一种：树形DP

也就是说如果我们知道了一个结点所有儿子结点的答案，我们是否可以维护出该结点的答案？也就是知否可以向上合并。

很明显不行，如果根节点的能力值比其中一个儿子结点的能力值弱，那么得到的答案就是错误的。

树上问题

第二种：树上问题转区间问题

对于子树询问，有一种很简单的办法就是用dfs序，就把子树问题转换成了一段连续的区间，当然树剖也可以，，但是这里没必要这么复杂。

当我们用dfs序把树上问题转换成区间问题后，题目所求就变成了求解区间 $[l, r]$ 中能力值大于 x 且忠诚度最高的人。

树上分块

该题可以用线段树求解，如果想不到线段树如何求解，可以考虑分块来做。

我们可以把得到的序列分成 $\sqrt{n}$ 块，对于每一个完整的块，我们可以按照能力值排序，这样我们可以快速的通过二分查找找到能力值 x ，那么一个块之内后面的数都是能力值 $>x$ 的。接下来需要如何找忠诚度最高的，其实也很简单，维护一个块内关于忠诚度的最大后缀就可以了。那么我们二分查找到的位置的最大后缀就是答案。对于首尾不完整的块，暴力查找即可。

树上分块

但是这种分块的做法效率是比较低的，时间复杂度最坏可以到 $\sqrt{n} * \log n * n$ 。

而该题可以直接用线段树来做，只是用线段树做只能离线处理。因为我们查找区间的时候查找的是能力值比 x 大的忠诚度最大的人。所以我们要想办法去除掉能力值小于 x 的人的影响，所以我们可以把每个人按照能力值从大到小排序。这样当我们在线段树上查询的时候，线段树上已经存在的值都是能力值 $\geq$ 当前值的数。

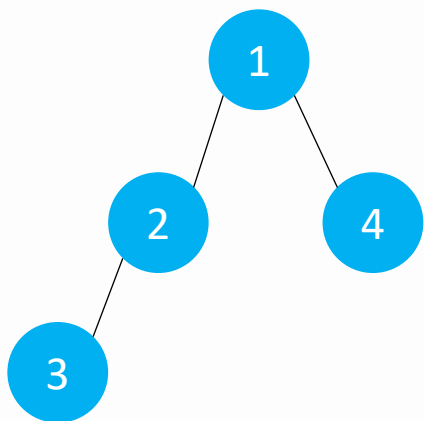
时间复杂度为 $n \log n$ 。

树上分块

在树上按照dfs序分块，一般用来处理子树查询问题，但是效率不高，而且一般都可以用 $\log n$ 的数据结构来替代，实用型不是很强，这只是一种树上分块的方法。

按照dfs序分块，可以保证块的大小和数量(每块大小一样，块的数量相对合理)，但是不能保证块的连通性。

比如在同一个块内的点不一定联通。



分成两个块，分别为 $[1, 2]$, $[3, 4]$ ，我们会发现单独看一个块，如 $[3, 4]$ ，他们并不是相连的，所以这种分块方法对于解决连通性问题不适用。

树上分块

如果题目中需要保证分的块联通，那么我们就需要换一种分块方法，我们前面是按照深度优先搜索的办法分块，因为深搜的性质，我们分的块可能是不连通的。

那么我们按照广搜的方式来遍历树并分块。我们从根节点开始遍历，对于某一个节点，如果其父亲节点所在块小于 $\sqrt{n}$ ，那么就把该节点加入其父亲节点所在块，否则就新建一个块。这样就能保证一个块内的节点都是与其父亲节点相连的，也就是连通的。

但是这样分块可以保证块的大小，连通性，但是不能保证块的数量，如果出现菊花图，或者很多点的度特别大这种情况，那么就导致效率特别低。

所以这种按照size分块的办法也很少使用。

树上分块

题目描述

给定一个 n 个节点的树，每个节点上有一个整数， i 号点的整数为 val_i 。

有 m 次询问，每次给出 u', v ，您需要将其解密得到 u, v ，并查询 u 到 v 的路径上有多少个不同的整数。

解密方式： $u = u' \text{ xor } lastans$ 。

$lastans$ 为上一次询问的答案，若无询问则为 0。

输入格式

第一行有两个整数 n 和 m 。

第二行有 n 个整数。第 i 个整数表示 val_i 。

在接下来的 $n - 1$ 行中，每行包含两个整数 u, v ，描述一条边。

在接下来的 m 行中，每行包含两个整数 u', v ，描述一组询问。

输出格式

对于每个询问，一行一个整数表示答案。

树上分块

该题有点类似于HH的项链，但是讲的是莫队的做法，不过效率比较低，被卡了，后面讲解了主席树的做法做法只会就成功A掉了。

该题把序列上的问题转移到了树上来求解，我们没有办法直接使用莫队或者直接使用主席树来解决。

还是和以前一样的做法，我们遇到树上的问题尽量把树上的问题转换成区间问题来求解。

如果我们用树剖来对树进行剖分，那么得到的是多段不连续的区间，而对于没每一段区间我们都要用主席树来维护，那么就涉及到了主席树的合并，有没有我们不知道，关键是我们也不会啊。

树上分块

我们可以考虑树上分块，既然是求两点之间的链上的信息，我们可以按照链来分块，但是和树剖不一样得地方在于，我们可以按照深度来分块，把树上隔 $\sqrt{n}$ 的深度就分为一块。

这样分块可以保证块内任意两点的深度差不超过 $\sqrt{n}$ ，同时，块的数量不超过 $\sqrt{n}$ ，但是不能保证块内的个数不超过 $\sqrt{n}$ 。我们需要的是两点之间的简单路径，所以影响不大。

当然，如果树的深度很浅，那么这种做法就不太合理，我们可以选择 $\sqrt{\text{deep}}$ 来选点。

如果嫌麻烦也可以随机撒点，也就是任选 $\sqrt{n}$ 个点作为关键点，那么任意两个关键点之间的点就是上一个关键点中拥有的点。

树上分块

那么此时树上就有 $\sqrt{n}$ 个关键点，我们完全可以预处理出来任意两个关键点之间的点权的种类，这个很好处理，以每一个关键点为根，跑一遍dfs就可以得到答案，但是由于空间时间的关系，我们只能得到最后的答案，而不能保存这里两个关键点之间具体的点权有哪些。

这样我们就知道了任意两个关键点之间的简单路径的数的种类，即任意两个完整的块之间的数的种类。

接下来我们只需要求链的端点到该块的顶点的数的种类就可以了。现在最大的问题是，链的端点到该块的顶点的数的种类我们可以暴力，但是两者怎么合并呢？我们是不知道关键点到关键点之间的具体信息的。

树上分块

所以该题我们已经转换成如何判断一个点是否在一条路径上出现过，我们可以用主席树来求解。

我们先用主席树维护出根节点到任意一个点的颜色种类。很好维护，从根节点开始遍历，每遍历一个点，就其父亲节点的线段树的基础上新增一个节点构成一个新的线段树，这就是主席树在树上的应用。

对于 $u-v$ 这条路径，我们可以看做两个部分，我们舍其 lca 为 x ，那么就是 $u-x$ 和 $v-x$ ，我们可以用 u 的这棵线段树减去 x 的线段树就得到了 $u-x$ 的具体节点信息，同理， $v-x$ 这一段也是相同的做法。接下来就判断点是否在这两条路径上了。最坏时间复杂度为 $n * \log n * \sqrt{n}$ 。

王室联邦

题目描述

[展开](#)

“余”人国的国王想重新编制他的国家。他想把他的国家划分成若干个省，每个省都由他们王室联邦的一个成员来管理。

他的国家有 N 个城市，编号为 $1 \dots N$ 。

一些城市之间有道路相连，任意两个不同的城市之间有且仅有一条直接或间接的道路。

为了防止管理太过分散，每个省至少要有 B 个城市。

为了能有效管理，每个省最多只有 $3 \times B$ 个城市。

每个省必须有一个省会，这个省会可以位于省内，也可以在该省外。

但是该省的任意一个城市到达省会所经过的道路上的城市（除了最后一个城市，即该省省会）都必须属于该省。

一个城市可以作为多个省的省会。

聪明的你快帮帮这个国王吧！



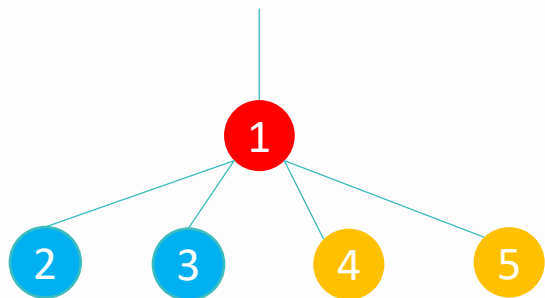
树上分块

该题是很多年前的一道四川省省选题，很裸的一道把树进行分块的题，并且需要分成的块满足给定的条件，后面常见的树上分块就是根据该题的做法研究出来的，叫做王室联邦分块。还有一些基于分块的算法，比如树上莫队也是基于王室联邦分块。



树上分块

该题要求块的数量在 $B-3B$ 之间，这个条件不难，主要是后面的一个条件，每一个块必须要有一个省会，这个省会可以不再块内，同时一个省会可以使多个块的省会，但是要求省内任意一个点到省会城市经过的点必须在块内。比如：



如果要求块的大小不超过2，那么可以2, 3一块，4, 5一块，并且这两块的省会都是1，这样是满足条件的。

树上分块

如果我们暂时还不知道如何分块，我们可以先考虑一些特殊的图，再推广到一般的情形。

(1) 菊花图(树)

n 个点，其中 $n-1$ 个点都以一个固定的点为中心

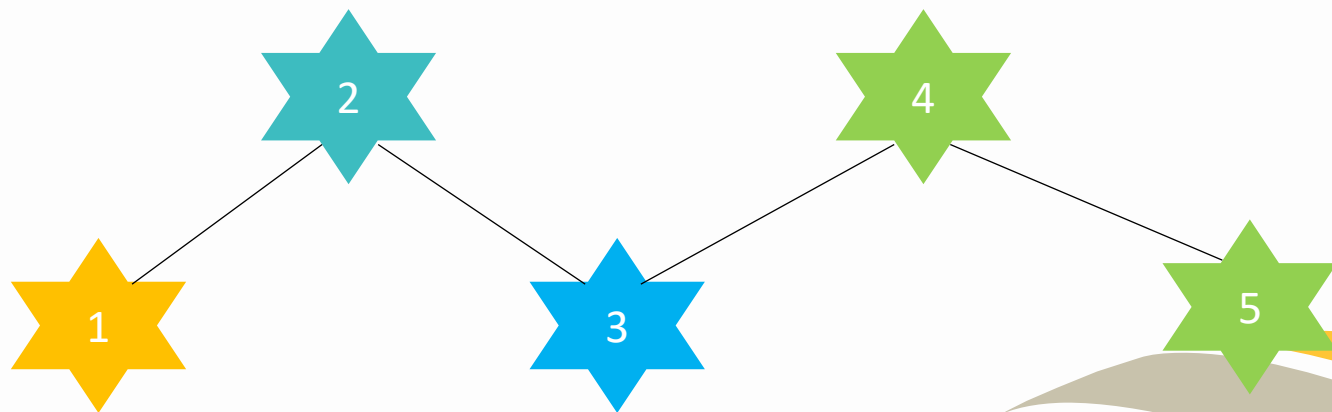
对于这种树，假设1为中心，我们完全可以对于每 $B-3B$ 个叶子节点化为一块，省会城市为1号节点，但是这样做会剩下不足一个块的就不满足要求，所以我们要仔细思考一下块的大小。

如果按照 k 来分块，并且分完之后还剩下 m 个节点，我们完全可以把 m 各节点放入最后一块中，那么就要保证 $m+k \leq 3B$ 。 $k \leq 1.5B$ ，当然为了方便，我们按照 B 来分块就可以了。

树上分块

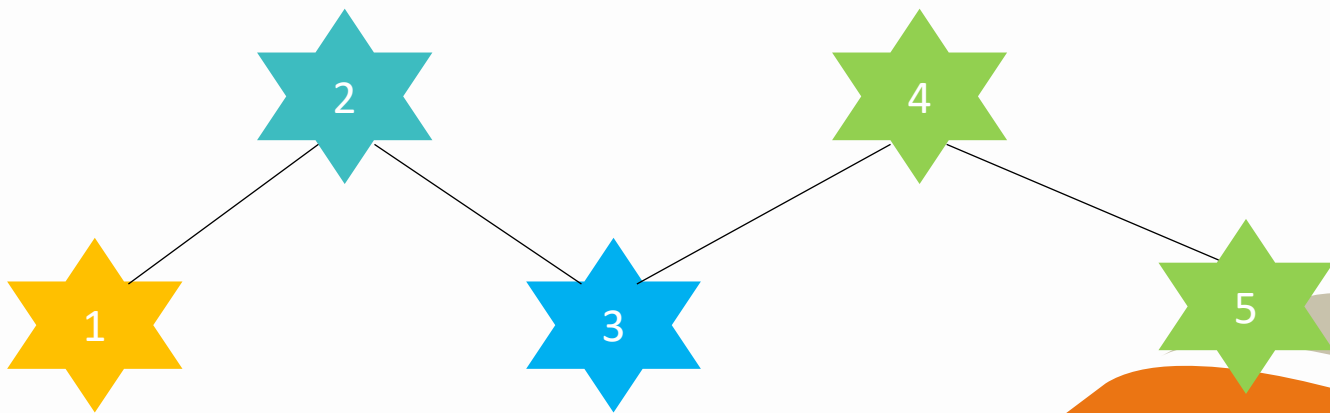
我们会发现对于菊花图的分块方法其实适用于任何树，我们可以把一棵树看成多个菊花图相连。但是问题在于对于任意一棵树，他的度很有可能小于 B ，那么我们单独对其中的一个菊花图进行分块很明显不满足要求。

那我们思考一下，假设有5个菊花图相连，并且每一个菊花图花瓣数量都小于 B ，并且总和大于 $3B$ 。



树上分块

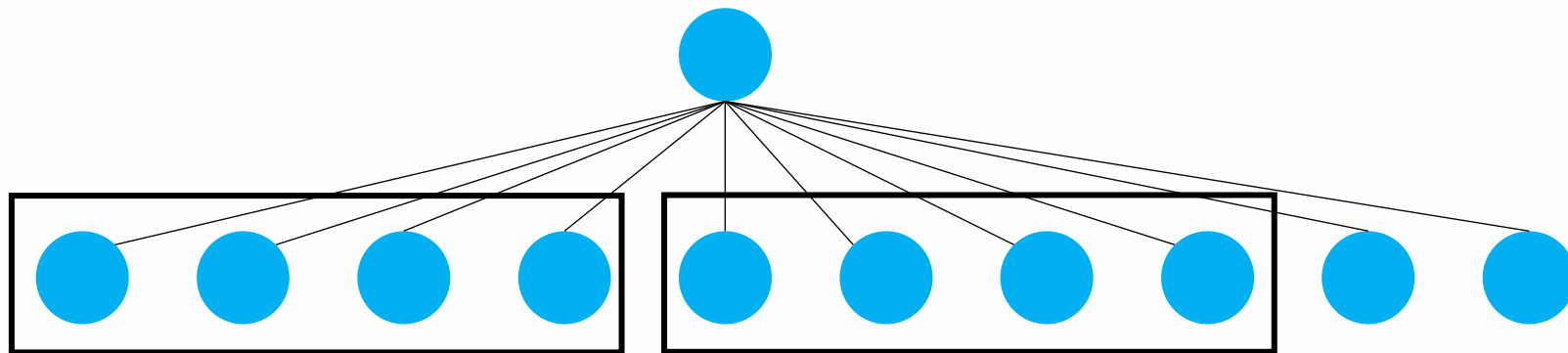
如果我们直接把1, 2放一起, 3, 4, 5放一起, 那么很有可能导致1, 2的总节点数不足B, 那么我们应该怎么分配呢? 我们可以先把1, 2放在一块, 如果不足, 再把3的部分加入, 那么应该加入3的哪一部分呢? 可能4, 5也是相同的情况, 那么我们应该先加叶子节点, 把根作为首都, 这样分块才满足条件。



树上分块

所以这里就有了一个树上分块的具体办法：

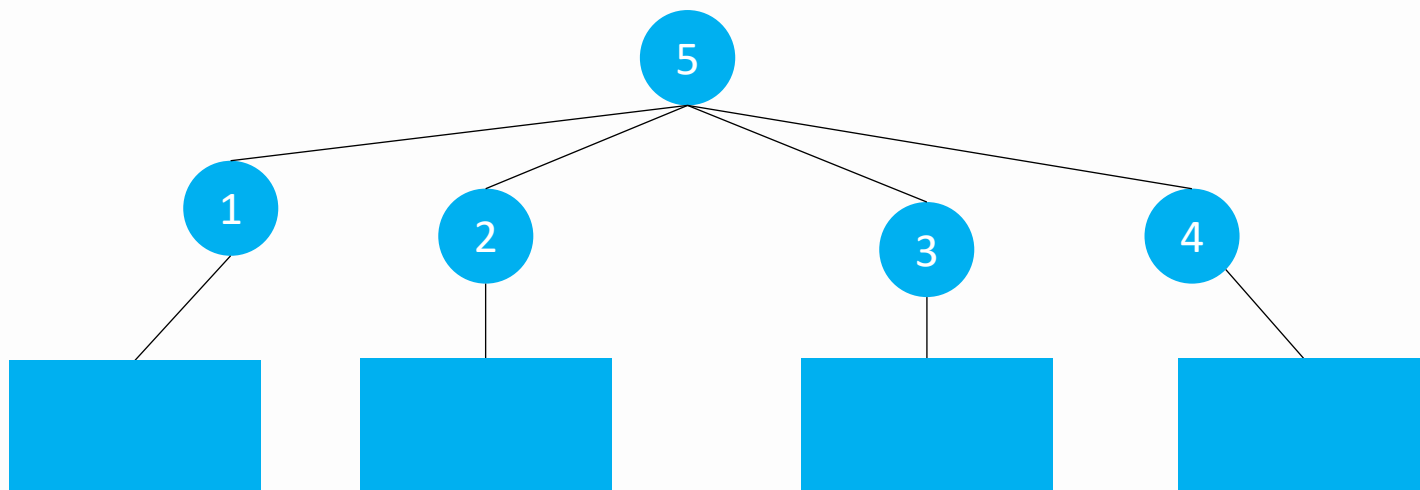
(1) 对于一棵子树，如果其叶子节点数量 $\geq b$ ，那么把这些叶子节点按照数量 b 来分块（根节点不分进去）



如果 $B=4$ ，那就可以把该子树先分成2个完整的块，剩下的两个节点和根节点先不分，存储起来。

树上分块

(2) 对于其上层的点，叶子节点的父亲节点更上层的点，对于他的每一个儿子，此时都是未被分块的，并且每棵子树中都可能含有 $1 \sim B$ 个未分配节点。那么我们完全可以继续合并，依次合并1, 2, 3, 4，如果数量大于 B ，则分为一块，5号节点为这些城市的省会。这里就不再是 B 了，因为此时每一块数量就小于等于 B



树上分块

(3) 然后按照这种方式继续向上分，每一块的大小都不会超过 $2B$ ，一个剩余的单独的子树中未分块的节点数不会超过 B ，最后一次分块完之后，如果还有剩余，也就是和根节点相连的点，可以把这两个合并，总数量不会超过 $3B$ 。

树上分块

```
void dfs(int u,int fa)
{
    int now=num;
    //now表示进入这棵子树时前面子树还未分块的节点数量
    //num表示此时已经遍历过的树中未分块的数量(从叶子节点向上遍历)
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            dfs(v,u);
            if(num-now>=m)
            //对于该棵子树而言, 如果其叶子节点数量>=m, 对叶子节点分块
            //注意此时前一棵子树中未分块的不参与进来
            //也就是说一棵子树中剩余的节点只与另一棵子树中剩余的节点组成一
            {
                root[++cnt]=u;//以u为根
                while(num>now)//存储块内信息, 这里用临时栈存储
                bl[s[num--]]=cnt;//栈内的元素属于该块
            }
        }
    }
    s[++num]=u;//当走出这棵子树的时候, 把根节点入栈。
    //栈的作用就是用来存储已经遍历过的还未被分块的节点
}
```